ObjectPoolSystem（对象池系统）

一、前言

早前我在引擎层嵌入了对象池系统：

① 对象池基础功能

- 自动扩容
- 自动缩容

② 高度解耦

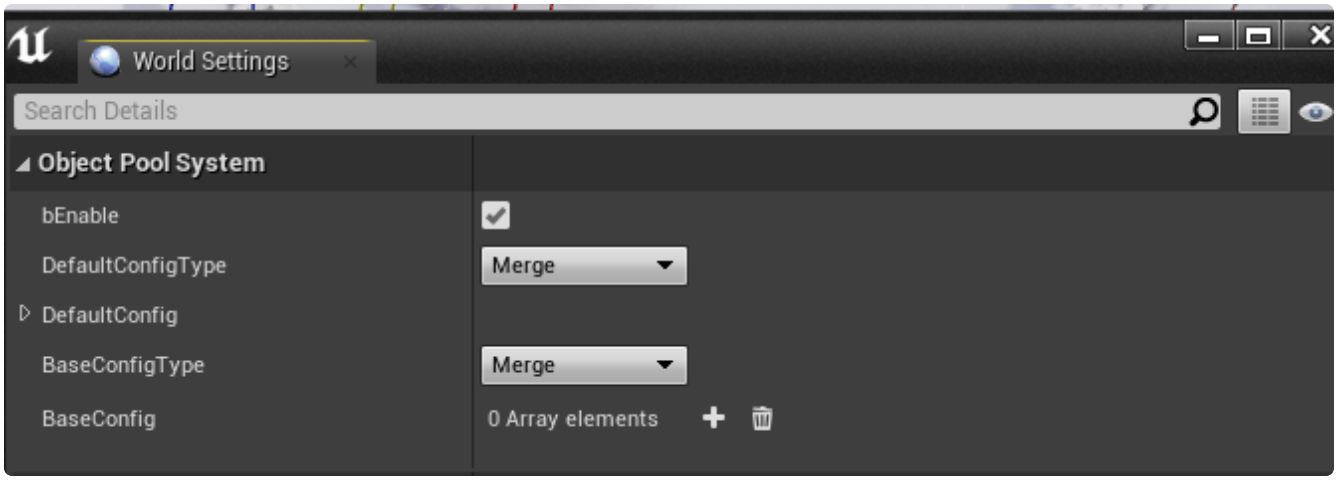
面向接口，并直接使用 SpawnActor 创建和 DestroyActor 销毁，业务层无需关心对象池系统实现细节

③ 支持对象池网络同步

目前 ds 和 client 都使用了对象池，因此也全面支持了对象池网络同步

④ WorldSettings 独立配置（近期优化）

引擎层对象池系统做了优化：支持在 WorldSettings 中为关卡配置独立的对象池

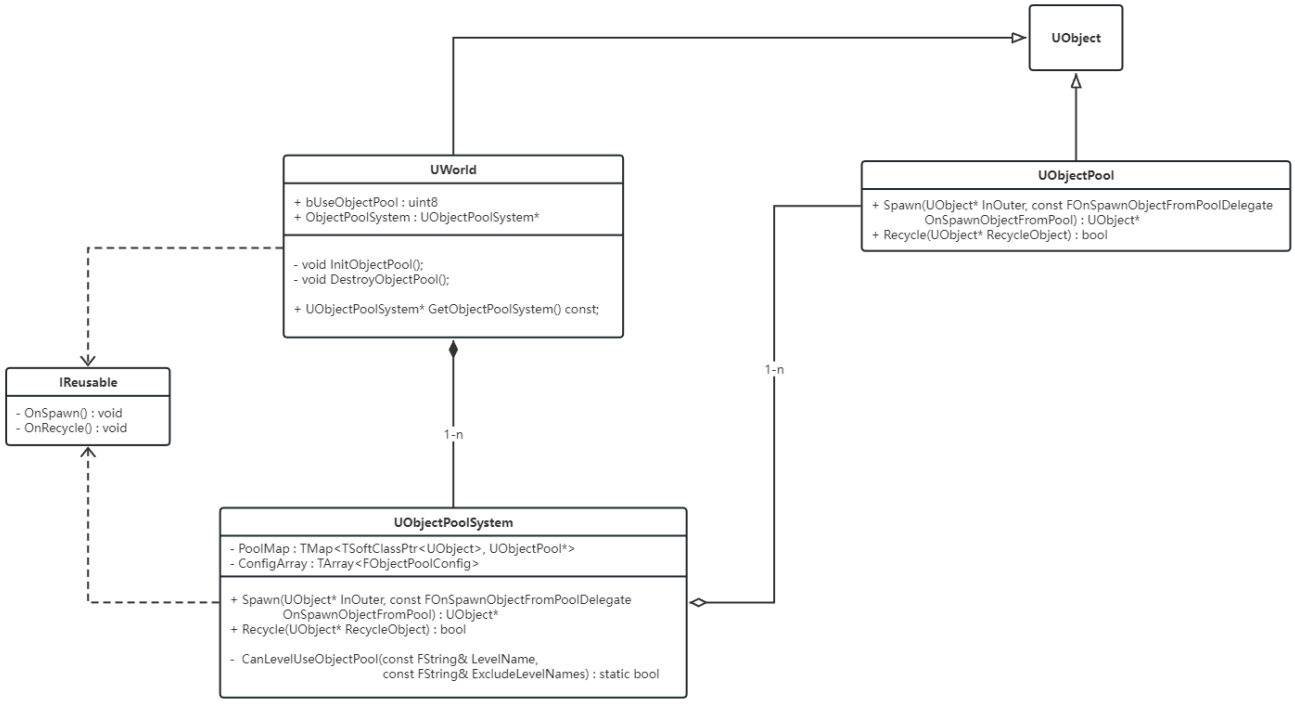


这样特定玩法才有的蓝图或者希望覆盖 ini 默认数据，便可在 WorldSettings 中配置，保证特定对象池的独立。

> 按照以前，由于ini是全局配置，所以会导致所有的关卡都会创建对应的对象池，这就有些浪费内存

二、框架说明

2.1 UML



2.2 IReusable

所有使用对象池的 class 都需要继承并实现接口 IReusable

C/C++

```cpp
class ENGINE_API IReusable
{
    GENERATED_IINTERFACE_BODY()

protected:
    /**
     * @brief [BlueprintNativeEvent] Called when object is spawning from Object Pool
     */
    UFUNCTION(BlueprintNativeEvent)
    void OnSpawn();

    /**
     * @brief [BlueprintNativeEvent] Called when object is recycling to Object Pool
     */
    UFUNCTION(BlueprintNativeEvent)
    void OnRecycle();
};
```

- **Actor**

对于 AActor 子类要使用对象池，只需要在 C++ 实现 IReusable 接口，并重载 OnSpawn 和 OnRecycle，后续就和普通 Actor 一样调用 SpawnActor 和 DestroyActor 即可

- **Object**

对于 UObject 子类要使用对象池，除了需要在 C++ 实现 IReusable 接口、并重载 OnSpawn 和 OnRecycle，还需自主调用 UKismetSystemLibrary::GetObjectPoolSystem 获取对象池实例，并调用 Spawn 和 Recycle

C/C++

```cpp
UObjectPoolSystem* UKismetSystemLibrary::GetObjectPoolSystem(const UObject* WorldContext)
```

2.2.1 OnSpawn

生成对象的回调，通常处理初始化对象相关的操作，功能上取代 BeginPlay

2.2.2 OnRecycle

回收对象的回调，通常处理重置对象相关的操作，功能上取代 BeginDestroy

2.2.3 生命周期

SpawnActor => NewObject => OnSpawn => BeginPlay（仅在第一次创建 Actor 时调用）

DestroyActor => OnRecycle => BeginDestroy（仅在对象池销毁后调用）

2.2.4 接口使用规范

由于对象池系统使用了 Cast，UE 中 Cast Interface 要求 class 必须是 native，

因此**所有使用对象池的 class 都必须有 native base class，不能直接从蓝图实现 IReusable 接口**

详情见引擎源码：*UObjectBaseUtility::GetInterfaceAddress*

C/C++

```cpp
void* UObjectBaseUtility::GetInterfaceAddress( UClass* InterfaceClass )
{
    void* Result = NULL;

    if ( InterfaceClass != NULL && InterfaceClass->HasAnyClassFlags(CLASS_Interface) && InterfaceClass != UInterface::StaticClass() )
    {
        // Script interface
        if ( !InterfaceClass->HasAnyClassFlags(CLASS_Native) )
        {
            if ( GetClass()->ImplementsInterface(InterfaceClass) )
            {
                // if it isn't a native interface, the address won't be different
                Result = this;
            }
        }
        // Native interface
        else
        {
            for( UClass* CurrentClass=GetClass(); Result == NULL && CurrentClass != NULL; CurrentClass = CurrentClass->GetSuperClass() )
            {
                for (TArray<FImplementedInterface>::TIterator It(CurrentClass->Interfaces); It; ++It)
                {
                    // See if this is the implementation we are looking for, and it was done natively, not in K2
                    FImplementedInterface& ImplInterface = *It;
                    if ( !ImplInterface.bImplementedByK2 && ImplInterface.Class->IsChildOf(InterfaceClass) )
                    {
                        Result = (uint8*)this + It->PointerOffset;
                        break;
                    }
                }
            }
        }
```

```
    }

    return Result;
}
```

2.3 网络同步

细节较多，感兴趣的可以参阅我写的知乎 blog：

UE4 引擎底层嵌入通用对象池系统（ObjectPoolSystem）

二、数据配置

下面简答说明一下最新数据配置的细节

2.1 配置数据类型说明

2.1.1 Default Config

默认配置，即没有配置对象池的类型在动态创建时默认使用的配置数据

> 通常不建议这样，因为不配置对象池的类型会在运行时创建对象池，这可能会导致big junk

C/C++

```cpp
USTRUCT(BlueprintType)
struct ENGINE_API FObjectPoolDefaultConfig
{
    GENERATED_BODY()

    // 对象池创建时默认容量
    UPROPERTY(EditAnywhere, Category=ObjectPoolDefaultConfig, meta=(UIMin=1, ClampMin=1, ToolTip="Default capacity for unconfigured class type, no less than 1"))
    int32 DefaultCapacity;

    // 最小容量
    UPROPERTY(EditAnywhere, Category=ObjectPoolDefaultConfig, meta=(UIMin=1, ClampMin=1, ToolTip="Minimal capacity, no less than 1"))
    int32 MinCapacity;

    // 扩容时的容量增长比例
    UPROPERTY(EditAnywhere, Category=ObjectPoolDefaultConfig, meta=(UIMin=1, ClampMin=1, ToolTip="Minimal grow factor, no less than 1"))
    float MinGrowFactor;

    // 最小扩容容量
    UPROPERTY(EditAnywhere, Category=ObjectPoolDefaultConfig, meta=(UIMin=1, ClampMin=1, ToolTip="Minimal grow size, no less than 1"))
    int32 MinGrowSize;

    // 自动缩减容量的CD(小于等于0, 表示不自动缩减; 反之表示缩减容量的CD)
    UPROPERTY(EditAnywhere, Category=ObjectPoolDefaultConfig, meta=(UIMin=0, ClampMin=0, ToolTip="Minimal auto reduce time"))
    float MinAutoReduceTime;

    // 是否需要在缩减容量后触发gc
    UPROPERTY(EditAnywhere, Category=ObjectPoolDefaultConfig, meta=(ToolTip="Whether force gc after reduce object pool size, be careful to do this for it can lag performance!"))
    bool ForceGCAfterReduce;
};
```

2.1.2 Base Config

基础数据，即为特定类型对象池配置用的

C/C++

```
USTRUCT(BlueprintType)
struct ENGINE_API FObjectPoolConfig
{
    GENERATED_BODY()

    // 类型
    UPROPERTY(EditAnywhere, Category=ObjectPoolConfig, meta=(MustImplement=Reusable, ToolTip="Object class type, must implements IReusable"))
    TSoftClassPtr<UObject> ClassType;

    // 初始容量
    UPROPERTY(EditAnywhere, Category=ObjectPoolConfig, meta=(UIMin=1, ClampMin=1, ToolTip="Object pool init capacity, no less than 1"))
    int32 InitCapacity = 4;

    // 增长比例
    UPROPERTY(EditAnywhere, Category=ObjectPoolConfig, meta=(UIMin=1.1, ClampMin=1.1, ToolTip="Object pool grow factor, no less than 1"))
    float GrowFactor = 1.4f;

    // 自动缩减容量的CD(小于等于0, 表示不自动缩减; 反之表示缩减容量的CD)
    UPROPERTY(EditAnywhere, Category=ObjectPoolConfig, meta=(UIMin=0, ClampMin=0, ToolTip=">0, this is auto reduce time(s) ; <= 0, close auto reduce function"))
    float AutoReduceTime = 0.f;

    // 剔除的关卡名, 及不创建该对象池的关卡(以;分割, 例如"Entry;MainMap"), 只在ini中起效, worldsetttings中不需要
    UPROPERTY(Category=ObjectPoolConfig, VisibleDefaultsOnly)
    FString ExcludeLevelNames;
};

```

2.1.3 Pool Config Type

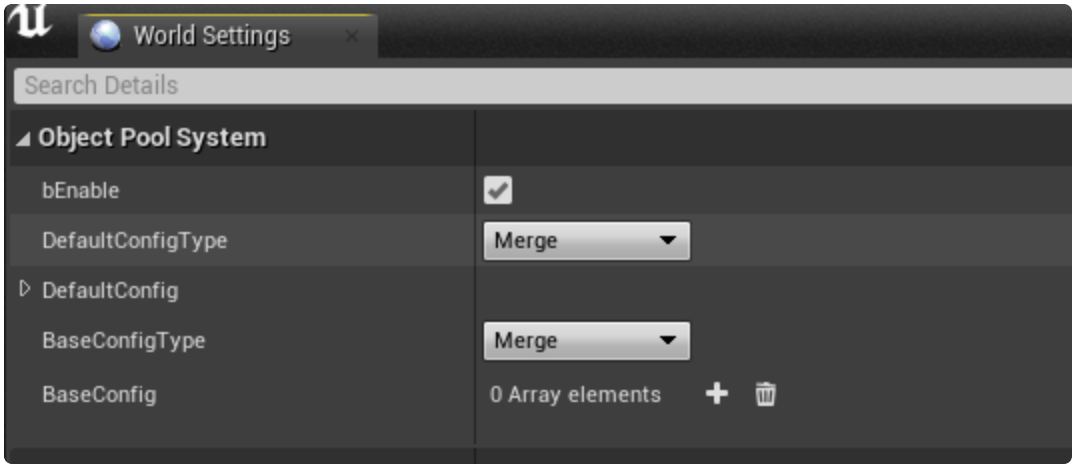
仅用于 worldsettings 中配置特殊需求的对象池数据，如果没有可以不用配置

- **OBCT_Merge**

合并 ini 和 wordsettings 的配置数据，但 wordsettings 可以覆盖 ini 的配置

> ① Default Config
> worldsetting不是默认值，就会使用worldsetting中的配置，否则使用ini配置
> ② Base Config
> 相同类型的对象池，以worldsetting为准；不同类型的对象池会合并

Defult Config 和 Base Config 默认都是 Merge



- **OBCT_UseIniOnly**

仅用 ini，忽略 worldsettings

- **OBCT_UseWorldSettingsOnly**

仅用 worldsettings，忽略 ini

C/C++

```
UENUM(BlueprintType)
enum EObjectPoolConfigType
{
    OBCT_Merge                    UMETA(DisplayName = "Merge", ToolTip = "Merge config between ini and worldsettings, worldsettings can override ini"),
    OBCT_UseIniOnly               UMETA(DisplayName = "UseIniOnly", ToolTip = "Use ini config only, ignore worldsettings"),
    OBCT_UseWorldSettingsOnly    UMETA(DisplayName = "UseWorldSettingsOnly", ToolTip = "Use worldsettings config only, ignore ini"),
};
```

2.2 Ini

ini 只配置通用的对象池，也就是大多数玩法都会创建的，例如基础技能、子弹、交互物等

Plain Text

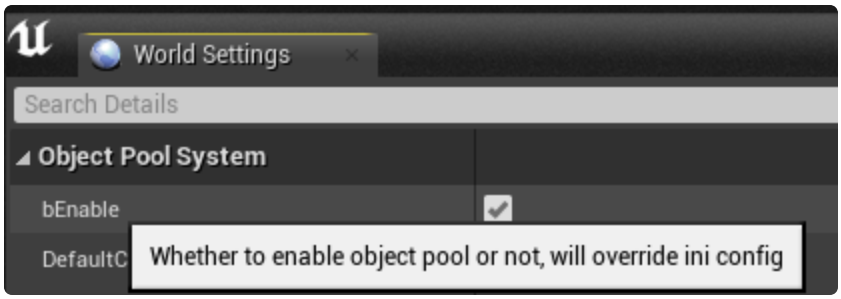
```
[/Script/Engine.ObjectPoolSystem]
;-----  default config data for ObjectPool  -----
+IniDefaultConfig=(DefaultCapacity=4, MinCapacity=4, MinGrowFactor=1.4, MinGrowSize=4, MinAutoReduceTime=0, ForceGCAfterReduce=false)

;-----  base config data for ObjectPool     -----
+IniBaseConfig=(ClassType="/Script/Game.FireBullet", InitCapacity=8, AutoReduceTime=0, ExcludeLevelNames="Entry;MainMap")
+IniBaseConfig=(ClassType="/Script/Game.Bullet", InitCapacity=40, AutoReduceTime=0, ExcludeLevelNames="Entry;MainMap")
+IniBaseConfig=(ClassType="/Script/Game.BulletBox", InitCapacity=20, AutoReduceTime=0, ExcludeLevelNames="Entry;MainMap")
```

2.3 WorldSettings

2.3.1 打开关闭对象池

对象池的开关也开放到 WorldSettings 中配置，默认都是打开的



同时动态的开关也是支持的

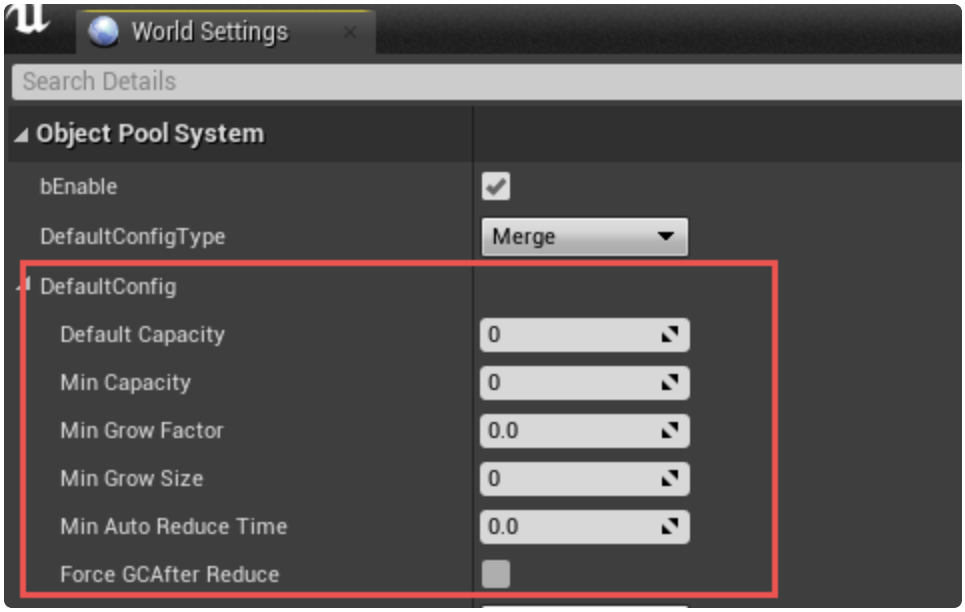
> 可以用于Runtime调试，也可用于动态关闭

C/C++

```
static bool GEnableObjectPool = true;
TAutoConsoleVariable<bool> CVarEnableObjectPool(
    TEXT("g.EnableObjectPool"),
    GEnableObjectPool,
    TEXT("Whether to enable object pool, true:enable ; false:disable"),
    ECVF_Default
);
```

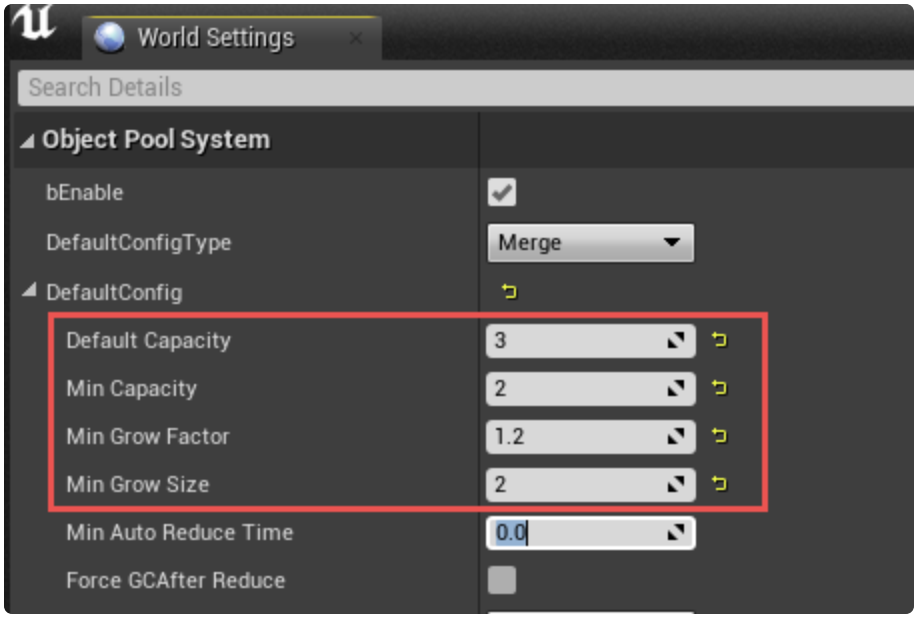
2.3.2 Default Config

> 默认采用Merge模式，且都是默认值，即使用ini配置数据



如果需要覆盖 ini 数据：

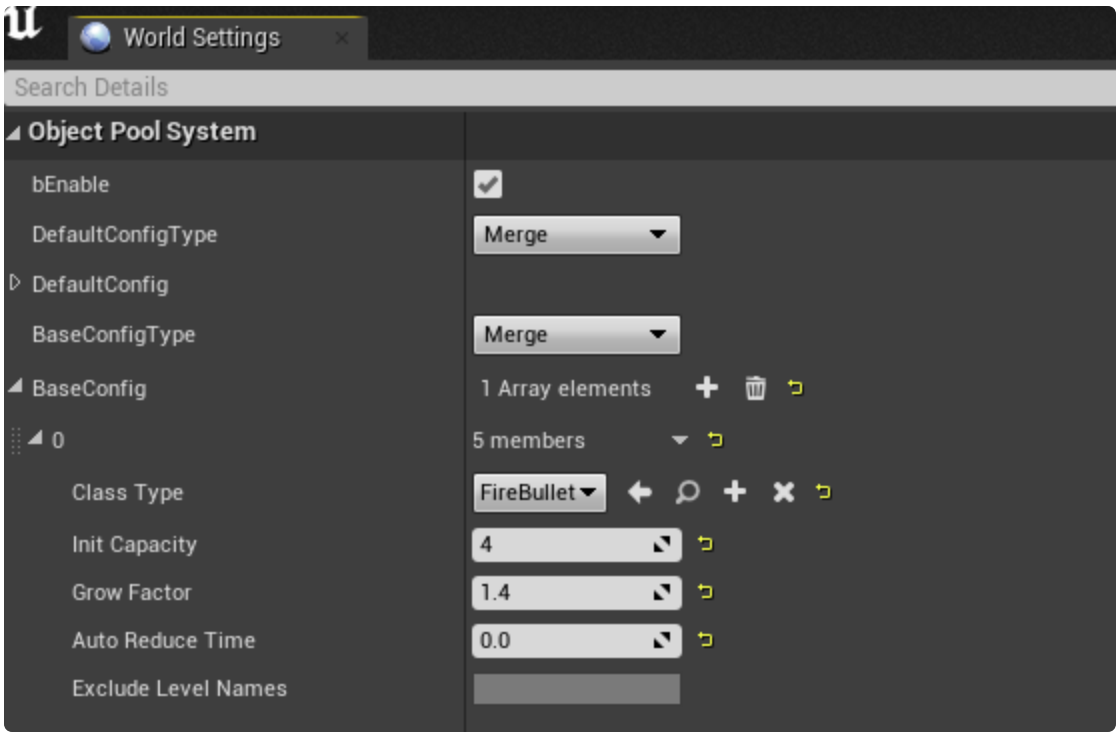
- 可以选择修改默认值



- 也可以选择 UseWorldSettingOnly

2.3.3 Base Config

针对特殊需求，WorldSettings 中配置特定玩法的对象池，例如特定的蓝图或者希望覆盖 ini 默认配置都可以



当然，没有特殊需求就不用配置 WorldSettings 了